

URI Pack v1 (PMV) — Archivos separados + Prompts para agente IA

Suposición: backend en **FastAPI + SQLAlchemy** con estructura `backend/app/`. Ajusta los imports si tu árbol difiere.

1) Archivos a crear

`app/models_shortlinks.py`

```
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, DateTime, func
from app.database import Base

class Shortlink(Base):
    __tablename__ = "shortlinks"
    code: Mapped[str] = mapped_column(String, primary_key=True)
    target_uri: Mapped[str] = mapped_column(String, nullable=False)
    gid: Mapped[str] = mapped_column(String, nullable=False, index=True)
    owner_id: Mapped[str | None] = mapped_column(String, nullable=True)
    created_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
```

`app/models_analytics.py`

```
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, DateTime, func, BigInteger
from app.database import Base

class AnalyticsEvent(Base):
    __tablename__ = "analytics_events"
    id: Mapped[int] = mapped_column(BigInteger, primary_key=True,
autoincrement=True)
    entity_gid: Mapped[str] = mapped_column(String, index=True)
    type: Mapped[str] = mapped_column(String) # "view" | "click"
    subresource: Mapped[str | None] = mapped_column(String)
    referrer: Mapped[str | None] = mapped_column(String)
    ua_hash: Mapped[str | None] = mapped_column(String)
    created_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
```

app/models_claims.py

```
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, DateTime, func
from app.database import Base

class EntityClaim(Base):
    __tablename__ = "entity_claims"
    entity_gid: Mapped[str] = mapped_column(String, primary_key=True)
    verify_token: Mapped[str] = mapped_column(String, nullable=False)
    status: Mapped[str] = mapped_column(String, default="pending") #
    pending|verified
    requested_by: Mapped[str | None] = mapped_column(String)
    verified_at: Mapped[DateTime | None] =
    mapped_column(DateTime(timezone=True))
    created_at: Mapped[DateTime] = mapped_column(DateTime(timezone=True),
    server_default=func.now())
```

app/schemas_pack.py

```
from pydantic import BaseModel, HttpUrl, Field
from typing import Optional, List

# Shortlinks
class ShortlinkCreate(BaseModel):
    gid: str = Field(..., description="GID del recurso")
    target_uri: HttpUrl

class ShortlinkOut(BaseModel):
    code: str
    url: str
    qr_png_url: str

# Analytics
class AnalyticsEventIn(BaseModel):
    entity_gid: str
    type: str # "view" | "click"
    subresource: Optional[str] = None
    referrer: Optional[str] = None

# Claim
class ClaimRequestIn(BaseModel):
    entity_gid: str
    requested_by: Optional[str] = None

class ClaimVerifyIn(BaseModel):
    entity_gid: str
    token: str
```

```
# JSON-LD
class LdResponse(BaseModel):
    jsonld: dict
```

app/utils/qr.py

```
import io
try:
    import qrcode
except ImportError:
    qrcode = None

def qr_png_bytes(url: str) -> bytes:
    if not qrcode:
        raise RuntimeError("Instala 'qrcode' (pip install qrcode[pil]) para generar PNGs")
    img = qrcode.make(url)
    buf = io.BytesIO()
    img.save(buf, format="PNG")
    return buf.getvalue()
```

app/utils/ua.py

```
import hashlib

def ua_fingerprint(ua: str, ip: str | None = None) -> str:
    s = (ua or "") + "|" + (ip or "")
    return hashlib.blake2b(s.encode(), digest_size=8).hexdigest()
```

app/utils/ld.py

```
def band_ld(band: dict) -> dict:
    return {
        "@context": "https://schema.org",
        "@type": "MusicGroup",
        "name": band.get("name"),
        "url": band.get("uri"),
        "sameAs": band.get("links") or []
    }

def album_ld(album: dict) -> dict:
    return {
        "@context": "https://schema.org",
        "@type": "MusicAlbum",
        "name": album.get("name"),
        "byArtist": {"@type": "MusicGroup", "name":
album.get("artist_name")},
```

```

        "url": album.get("uri"),
    }

```

app/routers/shortlinks.py

```

from fastapi import APIRouter, Depends, HTTPException, Response, status
from sqlalchemy.orm import Session
import hashlib, base64
from app.database import get_db
from app.schemas_pack import ShortlinkCreate, ShortlinkOut
from app.models_shortlinks import Shortlink
from app.utils.qr import qr_png_bytes

router = APIRouter(prefix="", tags=["Shortlinks"])

def _mk_code(gid: str, target_uri: str) -> str:
    h = hashlib.blake2b(f"{gid}|{target_uri}".encode(),
        digest_size=5).digest()
    return base64.urlsafe_b64encode(h).decode().rstrip("=") # ~8 chars

@router.post("/tools/shortlinks", response_model=ShortlinkOut)
def create_shortlink(payload: ShortlinkCreate, db: Session =
    Depends(get_db)):
    code = _mk_code(payload.gid, str(payload.target_uri))
    existing = db.query(Shortlink).filter(Shortlink.code == code).first()
    if existing:
        return ShortlinkOut(code=code, url=f"/s/{code}", qr_png_url=f"/tools/
shortlinks/{code}/qr.png")
    sl = Shortlink(code=code, target_uri=str(payload.target_uri),
        gid=payload.gid)
    db.add(sl)
    db.commit()
    return ShortlinkOut(code=code, url=f"/s/{code}", qr_png_url=f"/tools/
shortlinks/{code}/qr.png")

@router.get("/s/{code}")
def resolve_shortlink(code: str, db: Session = Depends(get_db)):
    sl = db.query(Shortlink).filter(Shortlink.code == code).first()
    if not sl:
        raise HTTPException(status_code=404, detail="Shortlink no
    encontrado")
    return Response(status_code=status.HTTP_302_FOUND, headers={"Location":
        sl.target_uri})

@router.get("/tools/shortlinks/{code}/qr.png")
def qr_png(code: str, db: Session = Depends(get_db)):
    sl = db.query(Shortlink).filter(Shortlink.code == code).first()
    if not sl:
        raise HTTPException(status_code=404, detail="Shortlink no
    encontrado")

```

```

png = qr_png_bytes(f"{sl.target_uri}")
return Response(content=png, media_type="image/png")

```

app/routers/embeds.py

```

from fastapi import APIRouter, Response, Request
from html import escape

router = APIRouter(prefix="/embed", tags=["Embeds"])

# /embed/{gid}/listen?
spotify=https://...&bandcamp=https://...&ytmusic=https://...&apple=https://...
@router.get("/{gid}/listen")
def listen(gid: str, request: Request):
    qp = request.query_params
    labels = [
        ("Spotify", "spotify"),
        ("Bandcamp", "bandcamp"),
        ("YouTube Music", "ytmusic"),
        ("Apple Music", "apple"),
    ]
    links = [(label, qp.get(key)) for label, key in labels if qp.get(key)]
    if not links:
        html = (
            "<div style='font:14px system-ui;padding:12px;border:1px solid #ddd;border-radius:10px'>"
            "<strong>Listen</strong><br/>No hay enlaces cargados aún.</div>"
        )
    else:
        btns = " ".join(
            f"<a href='{escape(u)}' target='_blank' rel='noopener' "
            f"style='padding:8px 12px;border:1px solid #ccc;border-radius:8px;text-decoration:none'>{escape(t)}</a>"
            for (t, u) in links
        )
        html = (
            "<div class='soup-embed-listen' style='font:14px system-ui;display:flex;gap:8px;"
            "align-items:center;border:1px solid #ddd;border-radius:12px;padding:10px'>"
            f"<span style='font-weight:600'>Listen:</span> {btns}</div>"
        )
    return Response(content=html, media_type="text/html")

```

app/routers/analytics.py

```

from fastapi import APIRouter, Request, Depends
from sqlalchemy.orm import Session
from sqlalchemy import text

```

```

from app.database import get_db
from app.schemas_pack import AnalyticsEventIn
from app.models_analytics import AnalyticsEvent
from app.utils.ua import ua_fingerprint

router = APIRouter(prefix="", tags=["Analytics"])

@router.post("/public/track")
async def track(evt: AnalyticsEventIn, request: Request, db: Session = Depends(get_db)):
    ua = request.headers.get("user-agent", "")
    ip = request.client.host if request.client else ""
    fp = ua_fingerprint(ua, ip)
    db.add(AnalyticsEvent(
        entity_gid=evt.entity_gid,
        type=evt.type,
        subresource=evt.subresource,
        referrer=evt.referrer,
        ua_hash=fp
    ))
    db.commit()
    return {"ok": True}

@router.get("/admin/analytics/summary")
def summary(gid: str, date_from: str, date_to: str, db: Session = Depends(get_db)):
    q = text(
        """
        SELECT date_trunc('day', created_at) AS d,
               sum(CASE WHEN type='view' THEN 1 ELSE 0 END) AS views,
               sum(CASE WHEN type='click' THEN 1 ELSE 0 END) AS clicks
        FROM analytics_events
        WHERE entity_gid = :gid AND created_at BETWEEN :df::timestampz
        AND :dt::timestampz
        GROUP BY 1 ORDER BY 1
        """
    )
    rows = db.execute(q, {"gid": gid, "df": date_from, "dt": date_to}).mappings().all()
    return [{"date": r["d"].isoformat(), "views": int(r["views"]), "clicks": int(r["clicks"])} for r in rows]

```

app/routers/claims.py

```

import secrets
from datetime import datetime, timezone
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.schemas_pack import ClaimRequestIn, ClaimVerifyIn

```

```

from app.models_claims import EntityClaim

router = APIRouter(prefix="", tags=["Claims"])

@router.post("/claim/request")
def claim_request(payload: ClaimRequestIn, db: Session = Depends(get_db)):
    token = secrets.token_urlsafe(12)
    existing = db.query(EntityClaim).filter(EntityClaim.entity_gid ==
payload.entity_gid).first()
    if existing:
        existing.verify_token = token
        existing.status = "pending"
    else:
        db.add(EntityClaim(entity_gid=payload.entity_gid, verify_token=token,
status="pending", requested_by=payload.requested_by))
        db.commit()
    return {
        "entity_gid": payload.entity_gid,
        "verify_token": token,
        "instructions": "Publica este token en una URL que controles. Luego
usa /claim/verify con el token."
    }

@router.post("/claim/verify")
def claim_verify(payload: ClaimVerifyIn, db: Session = Depends(get_db)):
    claim = db.query(EntityClaim).filter(EntityClaim.entity_gid ==
payload.entity_gid).first()
    if not claim:
        raise HTTPException(status_code=404, detail="Solicitud no
encontrada")
    if payload.token != claim.verify_token:
        raise HTTPException(status_code=400, detail="Token inválido")
    claim.status = "verified"
    claim.verified_at = datetime.now(timezone.utc)
    db.commit()
    return {"ok": True, "status": "verified"}

```

app/routers/public_ld.py

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session, Mapped, mapped_column
from sqlalchemy import String
from app.database import get_db, Base
from app.schemas_pack import LdResponse
from app.utils.ld import band_ld, album_ld
from app.models import Negocio, Producto

class EntityUriRegistry(Base):
    __tablename__ = "entity_uri_registry"
    gid: Mapped[str] = mapped_column(String, primary_key=True)

```

```

        tipo: Mapped[str] = mapped_column(String, index=True)           # "band" |
"album" | ...
        slug_actual: Mapped[str] = mapped_column(String)
        owner_table: Mapped[str] = mapped_column(String)              #
"negocios" | "productos"
        owner_id: Mapped[str] = mapped_column(String)

router = APIRouter(prefix="/public", tags=["Public JSON-LD"])

@router.get("/{tipo}/{gid}/ld.json", response_model=LdResponse)
def ld_json(tipo: str, gid: str, db: Session = Depends(get_db)):
    rec = db.query(EntityUriRegistry).filter(EntityUriRegistry.gid == gid,
EntityUriRegistry.tipo == tipo).first()
    if not rec:
        raise HTTPException(status_code=404, detail="No existe la entidad")
    if rec.owner_table == "negocios":
        band = db.query(Negocio).filter(Negocio.id == rec.owner_id).first()
        if not band:
            raise HTTPException(status_code=404, detail="Banda no
encontrada")
        data = {"name": band.nombre, "uri": f"https://soup.market/{tipo}/{
rec.slug_actual}", "links": []}
        return LdResponse(jsonld=band_ld(data))
    elif rec.owner_table == "productos":
        album = db.query(Producto).filter(Producto.id ==
rec.owner_id).first()
        if not album:
            raise HTTPException(status_code=404, detail="Álbum no
encontrado")
        artist = db.query(Negocio).filter(Negocio.id ==
album.negocio_id).first()
        data = {
            "name": album.nombre,
            "uri": f"https://soup.market/{tipo}/{rec.slug_actual}",
            "artist_name": artist.nombre if artist else None
        }
        return LdResponse(jsonld=album_ld(data))
    else:
        raise HTTPException(status_code=400, detail="Tipo no soportado en
PMV")

```

Incluir routers en `app/main.py`

```

from app.routers import shortlinks as shortlinks_router
from app.routers import embeds as embeds_router
from app.routers import analytics as analytics_router
from app.routers import claims as claims_router
from app.routers import public_ld as public_ld_router

# ...

```

```

app.include_router(shortlinks_router.router)
app.include_router(embeds_router.router)
app.include_router(analytics_router.router)
app.include_router(claims_router.router)
app.include_router(public_ld_router.router)

# Asegurar creación de tablas nuevas
from app import models_shortlinks, models_analytics, models_claims # noqa:
F401

```

(Opcional) SQL de migraciones rápidas

migrations/2025-08-12_uri_pack_v1.sql

```

CREATE TABLE IF NOT EXISTS shortlinks (
    code TEXT PRIMARY KEY,
    target_uri TEXT NOT NULL,
    gid TEXT NOT NULL,
    owner_id TEXT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS ix_shortlinks_gid ON shortlinks (gid);

CREATE TABLE IF NOT EXISTS analytics_events (
    id BIGSERIAL PRIMARY KEY,
    entity_gid TEXT NOT NULL,
    type TEXT NOT NULL,
    subresource TEXT NULL,
    referrer TEXT NULL,
    ua_hash TEXT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS ix_analytics_gid ON analytics_events (entity_gid);
CREATE INDEX IF NOT EXISTS ix_analytics_created ON analytics_events
(created_at);

CREATE TABLE IF NOT EXISTS entity_claims (
    entity_gid TEXT PRIMARY KEY,
    verify_token TEXT NOT NULL,
    status TEXT DEFAULT 'pending',
    requested_by TEXT NULL,
    verified_at TIMESTAMPTZ NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

2) Prompt maestro para agente de IA (aider / Continue / Code Assist)

Copia/pega este prompt en tu herramienta. El agente debe crear/editar archivos EXACTAMENTE como se indica y confirmar tests de humo al final.

Tarea: Implementar el URI Pack v1 (PMV) en un backend FastAPI+SQLAlchemy dentro de backend/app.

No modifiques contratos existentes salvo añadir rutas nuevas. Mantén compatibilidad.

1) Crea estos archivos con el contenido EXACTO que te paso (respetar imports):

- app/models_shortlinks.py
- app/models_analytics.py
- app/models_claims.py
- app/schemas_pack.py
- app/utils/qr.py
- app/utils/ua.py
- app/utils/ld.py
- app/routers/shortlinks.py
- app/routers/embeds.py (solo endpoint /embed/{gid}/listen)
- app/routers/analytics.py
- app/routers/claims.py
- app/routers/public_ld.py

2) Edita app/main.py para incluir routers:

include_router de shortlinks, embeds, analytics, claims, public_ld.

Importa models_shortlinks, models_analytics, models_claims antes de create_all.

3) (Opcional) Agrega migrations/2025-08-12_uri_pack_v1.sql con el SQL provisto y ejecútalo si hay sistema de migraciones.

4) Instala dependencia: `pip install qrcode[pil]`.

5) Pruebas de humo (no automatizadas):

- POST /tools/shortlinks {gid, target_uri} → devuelve {code, url, qr_png_url}
- GET /s/{code} → 302 a target_uri
- GET /tools/shortlinks/{code}/qr.png → image/png
- GET /embed/{gid}/listen?spotify=...&bandcamp=... → HTML con botones
- GET /public/{tipo}/{gid}/ld.json → JSON-LD válido para banda/álbum
- POST /public/track → {ok:true}; GET /admin/analytics/summary muestra agregados
- /claim/request y /claim/verify funcionan (estado verified)

6) Commits sugeridos (atómicos):

- feat(uri-pack): modelos shortlinks/analytics/claims
- feat(uri-pack): utils (qr, ua, ld)

- feat(uri-pack): routers shortlinks/embeds/analytics/claims/public_ld
- chore(uri-pack): wire-up main.py + deps

Condiciones:

- No romper endpoints existentes.
- Código limpio y pep8.
- Mensajes de error claros.

3) Comandos rápidos de verificación

```
# dependencia
pip install qrcode[pil]

# crear shortlink
curl -X POST http://localhost:8000/tools/shortlinks -H 'Content-Type: application/json' \
  -d '{"gid":"01J8ABC...", "target_uri":"https://soup.market/band/banda-nam~x7k3"}'

# redirección\ ncurl -i http://localhost:8000/s/XXXXXXXXX

# QR en navegador
http://localhost:8000/tools/shortlinks/XXXXXXXXX/qr.png

# embed listen
http://localhost:8000/embed/01J8ABC.../listen?spotify=https://open.spotify.com/...&bandcamp=https://...

# JSON-LD
http://localhost:8000/public/band/01J8ABC.../ld.json

# analytics
curl -X POST http://localhost:8000/public/track -H 'Content-Type: application/json' \
  -d '{"entity_gid":"01J8ABC...", "type":"view", "subresource":"listen"}'

curl "http://localhost:8000/admin/analytics/summary?gid=01J8ABC...&date_from=2025-08-01&date_to=2025-08-31"

# claim
curl -X POST http://localhost:8000/claim/request -H 'Content-Type: application/json' \
  -d '{"entity_gid":"01J8ABC...", "requested_by":"mephistofeles@band.com"}'

# usar token devuelto
curl -X POST http://localhost:8000/claim/verify -H 'Content-Type: application/json' \
  -d '{"entity_gid":"01J8ABC...", "token":"TOKEN"}'
```

Con esto el agente IA puede crear los archivos, cablearlos y dejar todo andando para el piloto. Si tu estructura no es `app/`, decime la ruta real y te ajusto los imports al toque.